

Lab Assignment 11.1

Name : Kathyayani Choudhari(46)

Roll No: 2303A53001

Task Description #1 – Stack Implementation

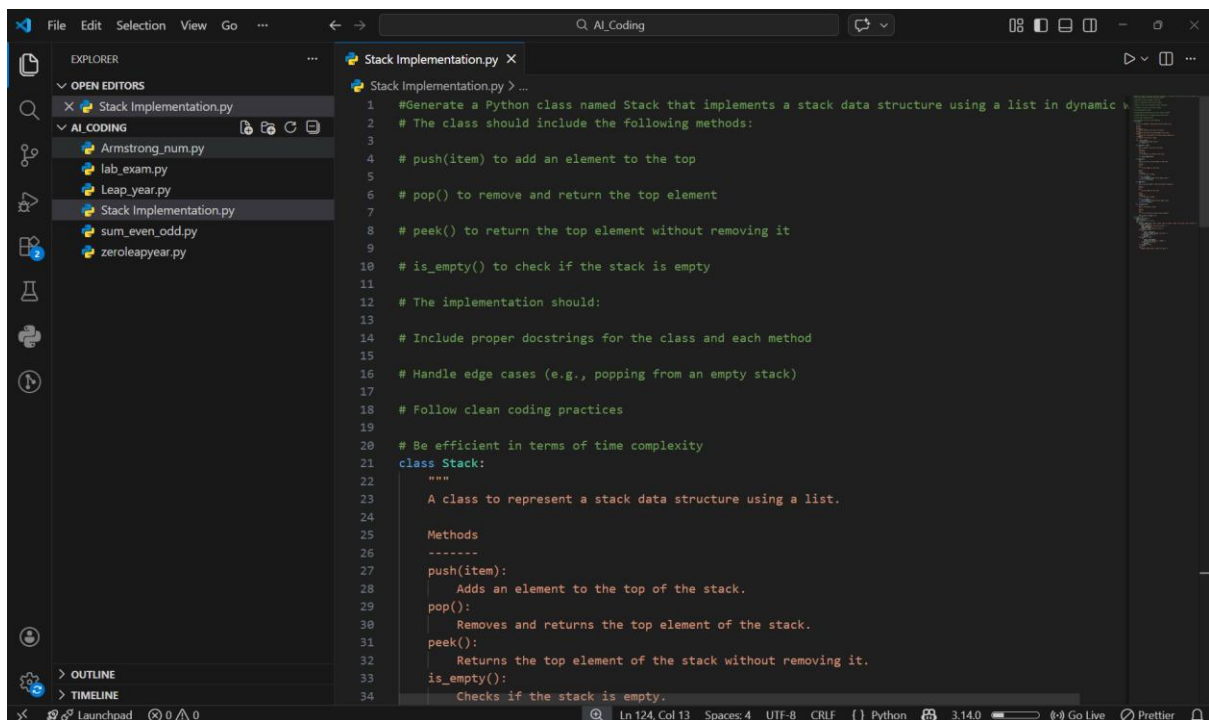
Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods.

Sample Input Code:

```
class Stack: pass
```

Expected Output: • A functional stack implementation with all required methods and docstrings.

Code:



```
1 #Generate a Python class named Stack that implements a stack data structure using a list in dynamic way
2 # The class should include the following methods:
3
4 # push(item) to add an element to the top
5
6 # pop() to remove and return the top element
7
8 # peek() to return the top element without removing it
9
10 # is_empty() to check if the stack is empty
11
12 # The implementation should:
13
14 # Include proper docstrings for the class and each method
15
16 # Handle edge cases (e.g., popping from an empty stack)
17
18 # Follow clean coding practices
19
20 # Be efficient in terms of time complexity
21 class Stack:
22     """
23     A class to represent a stack data structure using a list.
24
25     Methods
26     -----
27     push(item):
28         Adds an element to the top of the stack.
29     pop():
30         Removes and returns the top element of the stack.
31     peek():
32         Returns the top element of the stack without removing it.
33     is_empty():
34         Checks if the stack is empty.
```

```
Stack Implementation.py X
Stack Implementation.py > ...
21 class Stack:
34     """ Checks if the stack is empty.
35     """
36     def __init__(self):
37         """Initialize an empty stack."""
38         self.stack = []
39
40     def push(self, item):
41         """
42         Add an element to the top of the stack.
43
44         Parameters
45         -----
46         item : any
47             The element to be added to the stack.
48         """
49         self.stack.append(item)
50
51     def pop(self):
52         """
53         Remove and return the top element of the stack.
54
55         Returns
56         -----
57         any
58             The top element of the stack.
59
60         Raises
61         -----
62         IndexError
63             If the stack is empty.
64         """
65         if self.is_empty():
66             raise IndexError("Pop from an empty stack")
```

```
Stack Implementation.py > ...
21 class Stack:
51     def pop(self):
65         if self.is_empty():
66             raise IndexError("Pop from an empty stack")
67         return self.stack.pop()
68
69     def peek(self):
70         """
71         Return the top element of the stack without removing it.
72
73         Returns
74         -----
75         any
76             The top element of the stack.
77
78         Raises
79         -----
80         IndexError
81             If the stack is empty.
82         """
83         if self.is_empty():
84             raise IndexError("Peek from an empty stack")
85         return self.stack[-1]
86
87     def is_empty(self):
88         """
89         Check if the stack is empty.
90
91         Returns
92         -----
93         bool
94             True if the stack is empty, False otherwise.
95         """
96         return len(self.stack) == 0
```

```
Stack Implementation.py X
Stack Implementation.py > ...
21 class Stack:
87     def is_empty(self):
96         return len(self.stack) == 0
97 # Example usage:
98 if __name__ == "__main__":
99     stack = Stack()
100     #give dynamic input to stack
101     while True:
102         action = input("Enter 'push' to add, 'pop' to remove, 'peek' to view top, 'exit' to quit: ")
103         if action == 'push':
104             item = input("Enter the item to push: ")
105             stack.push(item)
106             print(f'{item}' pushed to stack.")
107         elif action == 'pop':
108             try:
109                 item = stack.pop()
110                 print(f'{item}' popped from stack.")
111             except IndexError as e:
112                 print(e)
113         elif action == 'peek':
114             try:
115                 item = stack.peek()
116                 print(f"Top element is: '{item}'.")
117             except IndexError as e:
118                 print(e)
119         elif action == 'exit':
120             break
121         else:
122             print("Invalid action. Please try again.")
123
124
```

Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
1
True
Pop from an empty stack
Peek from an empty stack
PS C:\Users\sonu\Desktop\AI_Coding> & C:/Python314/python.exe "c:/Users/sonu/Desktop/AI_Coding/Stack Implementation.py"
Enter 'push' to add, 'pop' to remove, 'peek' to view top, 'exit' to quit: push
Enter the item to push: 2
'2' pushed to stack.
Enter 'push' to add, 'pop' to remove, 'peek' to view top, 'exit' to quit: push
Enter the item to push: 3
'3' pushed to stack.
Enter 'push' to add, 'pop' to remove, 'peek' to view top, 'exit' to quit: peek
Top element is: '3'.
Enter 'push' to add, 'pop' to remove, 'peek' to view top, 'exit' to quit: pop
'3' popped from stack.
Enter 'push' to add, 'pop' to remove, 'peek' to view top, 'exit' to quit: peek
Top element is: '2'.
Enter 'push' to add, 'pop' to remove, 'peek' to view top, 'exit' to quit: 
```

Task Description #2 – Queue Implementation

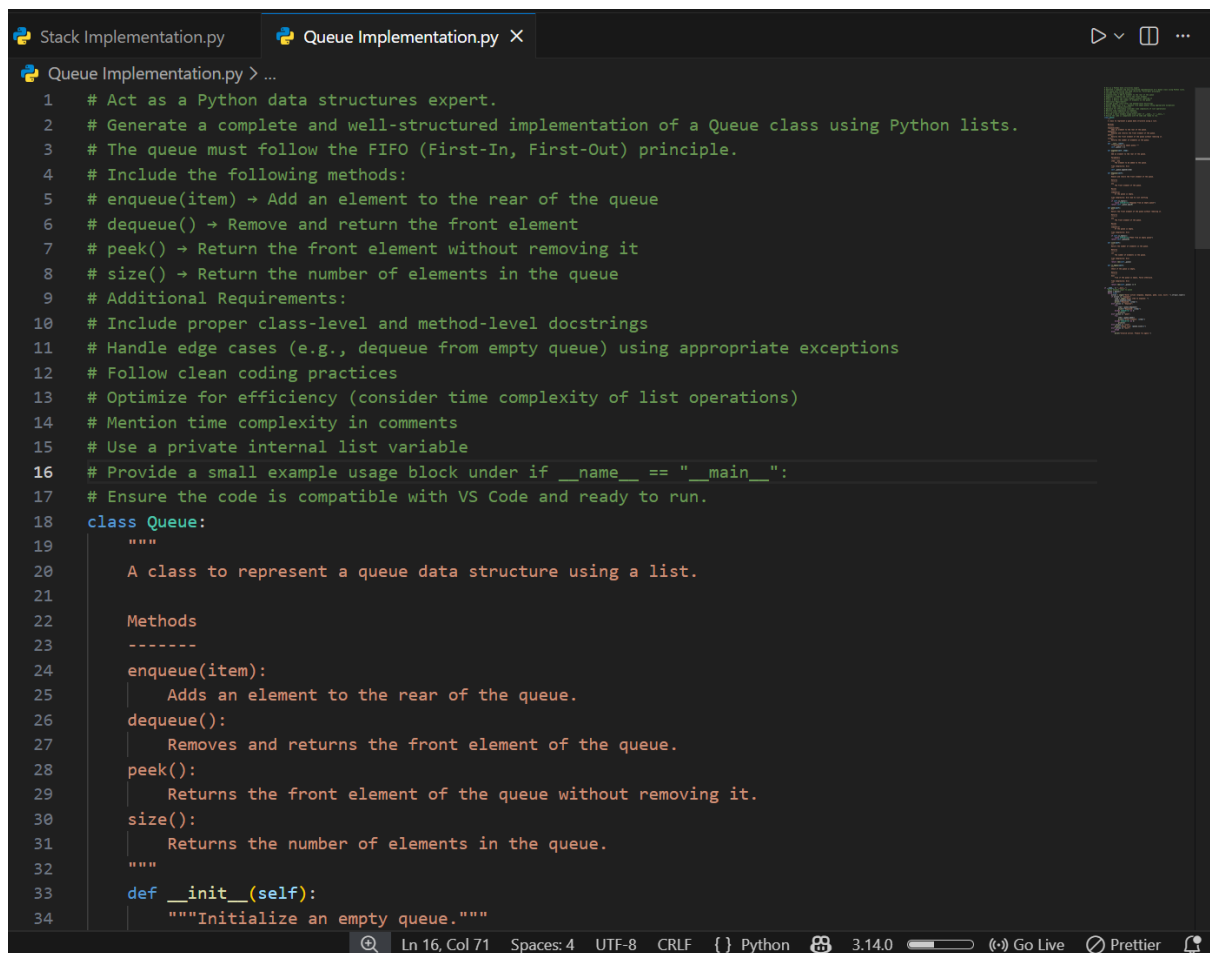
Task: Use AI to implement a Queue using Python lists.

Sample Input Code:

```
class Queue: pass
```

Expected Output: • FIFO-based queue class with enqueue, dequeue, peek, and size methods.

Code:



```
1  # Act as a Python data structures expert.
2  # Generate a complete and well-structured implementation of a Queue class using Python lists.
3  # The queue must follow the FIFO (First-In, First-Out) principle.
4  # Include the following methods:
5  # enqueue(item) → Add an element to the rear of the queue
6  # dequeue() → Remove and return the front element
7  # peek() → Return the front element without removing it
8  # size() → Return the number of elements in the queue
9  # Additional Requirements:
10 # Include proper class-level and method-level docstrings
11 # Handle edge cases (e.g., dequeue from empty queue) using appropriate exceptions
12 # Follow clean coding practices
13 # Optimize for efficiency (consider time complexity of list operations)
14 # Mention time complexity in comments
15 # Use a private internal list variable
16 # Provide a small example usage block under if __name__ == "__main__":
17 # Ensure the code is compatible with VS Code and ready to run.
18 class Queue:
19     """
20     A class to represent a queue data structure using a list.
21
22     Methods
23     -----
24     enqueue(item):
25         Adds an element to the rear of the queue.
26     dequeue():
27         Removes and returns the front element of the queue.
28     peek():
29         Returns the front element of the queue without removing it.
30     size():
31         Returns the number of elements in the queue.
32     """
33     def __init__(self):
34         """Initialize an empty queue."""
```

Stack Implementation.py Queue Implementation.py X

Queue Implementation.py > ...

```
18 class Queue:
33     def __init__(self):
34         """Initialize an empty queue."""
35         self._queue = []
36
37     def enqueue(self, item):
38         """
39         Add an element to the rear of the queue.
40
41         Parameters
42         -----
43         item : any
44             The element to be added to the queue.
45
46         Time Complexity: O(1)
47         """
48         self._queue.append(item)
49
50     def dequeue(self):
51         """
52         Remove and return the front element of the queue.
53
54         Returns
55         -----
56         any
57             The front element of the queue.
58
59         Raises
60         -----
61         IndexError
62             If the queue is empty.
63
64         Time Complexity: O(n) due to list shifting
65         """
```

Ln 16, Col 71 Spaces: 4 UTF-8 CRLF Python 3.14.0 Go Live Prettier

```
Stack Implementation.py Queue Implementation.py X
Queue Implementation.py > ...
18 class Queue:
50     def dequeue(self):
66         if self.is_empty():
67             raise IndexError("Dequeue from an empty queue")
68         return self._queue.pop(0)
69
70     def peek(self):
71         """
72         Return the front element of the queue without removing it.
73
74         Returns
75         -----
76         any
77             The front element of the queue.
78
79         Raises
80         -----
81         IndexError
82             If the queue is empty.
83
84         Time Complexity: O(1)
85         """
86         if self.is_empty():
87             raise IndexError("Peek from an empty queue")
88         return self._queue[0]
89
90     def size(self):
91         """
92         Return the number of elements in the queue.
93
94         Returns
95         -----
96         int
97             The number of elements in the queue.
```

Ln 16, Col 71 Spaces: 4 UTF-8 CRLF {} Python 3.14.0 Go Live Prettier

```
Stack Implementation.py Queue Implementation.py X
Queue Implementation.py > ...
18 class Queue:
90     def size(self):
91         """
92         Time Complexity: O(1)
93         """
94         return len(self._queue)
95
103     def is_empty(self):
104         """
105         Check if the queue is empty.
106
107         Returns
108         -----
109         bool
110         True if the queue is empty, False otherwise.
111
112         Time Complexity: O(1)
113         """
114         return len(self._queue) == 0
115
116 if __name__ == "__main__":
117     #give dynamic input to queue
118     queue = Queue()
119     while True:
120         action = input("Enter action (enqueue, dequeue, peek, size, exit): ").strip().lower()
121         if action == "enqueue":
122             item = input("Enter item to enqueue: ")
123             queue.enqueue(item)
124             print(f"Enqueued: {item}")
125         elif action == "dequeue":
126             try:
127                 item = queue.dequeue()
128                 print(f"Dequeued: {item}")
129             except IndexError as e:
130                 print(e)
131
132
Ln 16, Col 71 Spaces: 4 UTF-8 CRLF Python 3.14.0 Go Live Prettier
```

```
Stack Implementation.py Queue Implementation.py X
Queue Implementation.py > ...
113 class Queue:
114     def __init__(self):
115         self.queue = []
116     def __len__(self):
117         return len(self.queue) == 0
118
119 if __name__ == "__main__":
120     #give dynamic input to queue
121     queue = Queue()
122     while True:
123         action = input("Enter action (enqueue, dequeue, peek, size, exit): ").strip().lower()
124         if action == "enqueue":
125             item = input("Enter item to enqueue: ")
126             queue.enqueue(item)
127             print(f"Enqueued: {item}")
128         elif action == "dequeue":
129             try:
130                 item = queue.dequeue()
131                 print(f"Dequeued: {item}")
132             except IndexError as e:
133                 print(e)
134         elif action == "peek":
135             try:
136                 item = queue.peek()
137                 print(f"Front element: {item}")
138             except IndexError as e:
139                 print(e)
140         elif action == "size":
141             print(f"Queue size: {queue.size()}")
142         elif action == "exit":
143             break
144         else:
145             print("Invalid action. Please try again.")
```

Output:

```
14 # Mention time complexity in comments
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\sonu\Desktop\AI_Coding> & C:/Python314/python.exe "c:/Users/sonu/Desktop/AI_Coding/Queue Implementation.py"
Enter action (enqueue, dequeue, peek, size, exit): enqueue
Enter item to enqueue: 3
Enqueued: 3
Enter action (enqueue, dequeue, peek, size, exit): size
Queue size: 1
Enter action (enqueue, dequeue, peek, size, exit): enqueue
Enter item to enqueue: 4
Enqueued: 4
Enter action (enqueue, dequeue, peek, size, exit): peek
Front element: 3
Enter action (enqueue, dequeue, peek, size, exit): exit
PS C:\Users\sonu\Desktop\AI_Coding>
```


Task Description #3 – Linked List

Task: Use AI to generate a Singly Linked List with insert and display methods.

Sample Input Code:

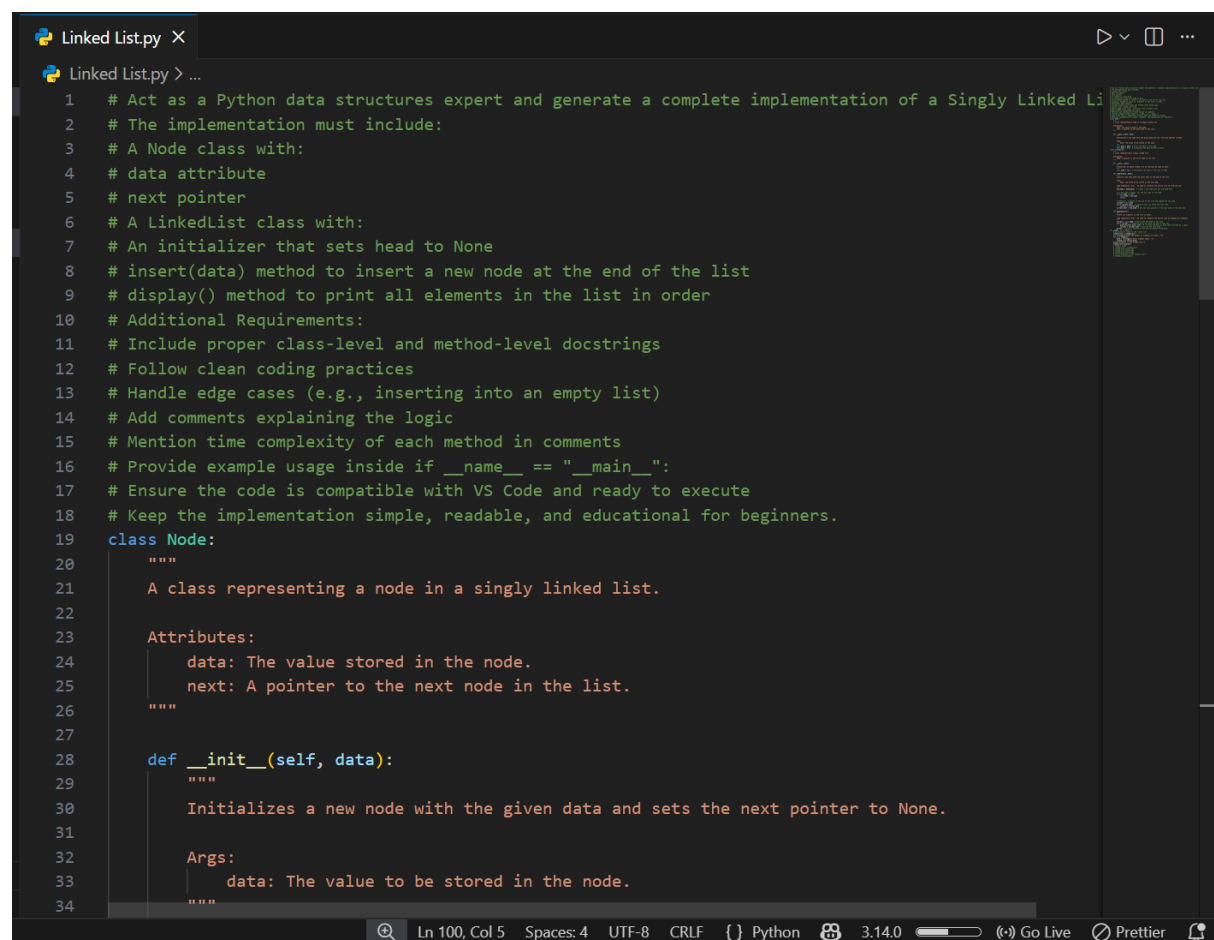
```
class Node: pass
```

```
class LinkedList: pass
```

Expected Output:

- A working linked list implementation with clear method documentation.

Code:



```
1  # Act as a Python data structures expert and generate a complete implementation of a Singly Linked List
2  # The implementation must include:
3  # A Node class with:
4  #   data attribute
5  #   next pointer
6  # A LinkedList class with:
7  #   An initializer that sets head to None
8  #   insert(data) method to insert a new node at the end of the list
9  #   display() method to print all elements in the list in order
10 # Additional Requirements:
11 # Include proper class-level and method-level docstrings
12 # Follow clean coding practices
13 # Handle edge cases (e.g., inserting into an empty list)
14 # Add comments explaining the logic
15 # Mention time complexity of each method in comments
16 # Provide example usage inside if __name__ == "__main__":
17 # Ensure the code is compatible with VS Code and ready to execute
18 # Keep the implementation simple, readable, and educational for beginners.
19 class Node:
20     """
21     A class representing a node in a singly linked list.
22
23     Attributes:
24         data: The value stored in the node.
25         next: A pointer to the next node in the list.
26     """
27
28     def __init__(self, data):
29         """
30         Initializes a new node with the given data and sets the next pointer to None.
31
32         Args:
33             data: The value to be stored in the node.
34         """
```

```
Linked List.py X
Linked List.py > ...
19 class Node:
20     def __init__(self, data):
35         self.data = data # Store the data in the node
36         self.next = None # Initialize the next pointer to None
37 class LinkedList:
38     """
39     A class representing a singly linked list.
40
41     Attributes:
42         head: A pointer to the first node in the list.
43     """
44
45     def __init__(self):
46         """
47         Initializes an empty linked list by setting the head to None.
48         """
49         self.head = None # Initialize the head of the list to None
50
51     def insert(self, data):
52         """
53         Inserts a new node with the given data at the end of the list.
54
55         Args:
56             data: The value to be stored in the new node.
57
58         Time Complexity: O(n) - We need to traverse the entire list to find the end.
59         """
60         new_node = Node(data) # Create a new node with the provided data
61
62         # If the list is empty, set the new node as the head
63         if self.head is None:
64             self.head = new_node
65         return
66
```

Ln 100, Col 5 Spaces: 4 UTF-8 CRLF Python 3.14.0 Go Live Prettier

```
Linked List.py X
Linked List.py > ...
37 class LinkedList:
51     def insert(self, data):
66
67         # Otherwise, traverse to the end of the list and append the new node
68         current = self.head
69         while current.next: # Traverse until we reach the last node
70             current = current.next
71         current.next = new_node # Set the next pointer of the last node to the new node
72
73     def display(self):
74         """
75         Prints all elements in the list in order.
76
77         Time Complexity: O(n) - We need to traverse the entire list to display all elements.
78         """
79         current = self.head # Start from the head of the list
80         while current: # Traverse until we reach the end of the list
81             print(current.data, end=' ') # Print the data of each node followed by a space
82             current = current.next # Move to the next node
83         print() # Print a newline at the end for better formatting
84 if __name__ == "__main__":
85     # give dynamic input to the linked list
86     linked_list = LinkedList()
87     n = int(input("Enter the number of elements to insert: "))
88     for i in range(n):
89         data = int(input(f"Enter element {i+1}: "))
90         linked_list.insert(data)
91     print("Elements in the linked list:")
92     linked_list.display()
93     # Example usage:
94     # linked_list = LinkedList()
95     # linked_list.insert(10)
96     # linked_list.insert(20)
97     # linked_list.insert(30)
```

Ln 100, Col 5 Spaces: 4 UTF-8 CRLF {} Python 3.14.0 Go Live Prettier

```
Linked List.py X
Linked List.py > ...
37 class LinkedList:
73     def display(self):
77         Time Complexity: O(n) - We need to traverse the entire list to display all elements.
78         """
79         current = self.head # Start from the head of the list
80         while current: # Traverse until we reach the end of the list
81             print(current.data, end=' ') # Print the data of each node followed by a space
82             current = current.next # Move to the next node
83             print() # Print a newline at the end for better formatting
84 if __name__ == "__main__":
85     # give dynamic input to the linked list
86     linked_list = LinkedList()
87     n = int(input("Enter the number of elements to insert: "))
88     for i in range(n):
89         data = int(input(f"Enter element {i+1}: "))
90         linked_list.insert(data)
91     print("Elements in the linked list:")
92     linked_list.display()
93     # Example usage:
94     # linked_list = LinkedList()
95     # linked_list.insert(10)
96     # linked_list.insert(20)
97     # linked_list.insert(30)
98     # print("Elements in the linked list:")
99     # linked_list.display()
100
```

Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\sonu\Desktop\AI_Coding> & C:/Python314/python.exe "c:/Users/sonu/Desktop/AI_Coding/Linked List.py"
Enter the number of elements to insert: 2
Enter element 1: 3
Enter element 2: 4
Elements in the linked list:
3 4
PS C:\Users\sonu\Desktop\AI_Coding>
```

Task Description #4 – Binary Search Tree (BST)

Task: Use AI to create a BST with insert and in-order traversal methods.

Sample Input Code:

```
class BST: pass
```

Expected Output: • BST implementation with recursive insert and traversal methods.

Code:

```
Binary Search Tree (BST).py X
Binary Search Tree (BST).py > ...
1  # Act as a Python data structures expert.
2  # Generate a complete implementation of a Binary Search Tree (BST) in Python.
3  # The implementation must:
4  # Define a BST class
5  # Use a separate internal Node structure (with data, left, and right attributes)
6  # Implement a recursive insert(value) method that follows BST properties:
7  # Left subtree contains smaller values
8  # Right subtree contains larger values
9  # Implement a recursive inorder_traversal() method that prints elements in sorted order
10 # Additional Requirements:
11 # Use recursion for both insertion and traversal
12 # Include proper class-level and method-level docstrings
13 # Handle edge cases (e.g., inserting into an empty tree)
14 # Add comments explaining the recursive logic
15 # Mention time complexity in comments (Best, Average, Worst case)
16 # Follow clean coding practices
17 # Include example usage under if __name__ == "__main__":
18 # Ensure the code runs correctly in VS Code
19 # Keep the implementation beginner-friendly but conceptually accurate.
20 class Node:
21     """
22     A Node in a Binary Search Tree (BST).
23
24     Attributes:
25         data (int): The value stored in the node.
26         left (Node): Reference to the left child node.
27         right (Node): Reference to the right child node.
28     """
29
30     def __init__(self, data):
31         """
32         Initializes a new Node with the given data.
33
34         Args:
```

```
Binary Search Tree (BST).py X
Binary Search Tree (BST).py > ...
20 class Node:
30     def __init__(self, data):
34         Args:
35             data (int): The value to be stored in the node.
36         """
37         self.data = data
38         self.left = None
39         self.right = None
40
41 class BinarySearchTree:
42     """
43     A Binary Search Tree (BST) implementation.
44
45     Methods:
46         insert(value): Inserts a value into the BST.
47         inorder_traversal(): Prints the elements of the BST in sorted order.
48     """
49
50     def __init__(self):
51         """
52         Initializes an empty Binary Search Tree.
53         """
54         self.root = None
55
56     def insert(self, value):
57         """
58         Inserts a value into the BST while maintaining the BST properties.
59
60         Args:
61             value (int): The value to be inserted into the BST.
62
63         Time Complexity:
64             Best Case:  $O(\log n)$  - when the tree is balanced
65             Average Case:  $O(\log n)$  - when the tree is reasonably balanced
66             Worst Case:  $O(n)$  - when the tree is skewed
```

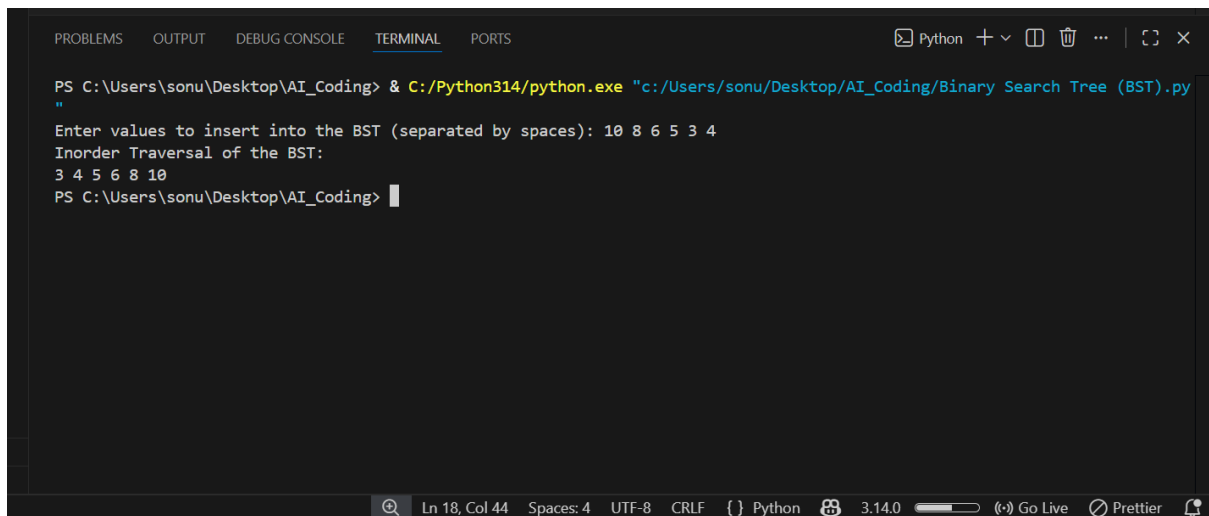
```
Binary Search Tree (BST).py X
Binary Search Tree (BST).py > ...
41 class BinarySearchTree:
56     def insert(self, value):
57         """
58         Worst case:  $O(n)$  - when the tree becomes a linked list (unbalanced)
59         """
60         if self.root is None:
61             # If the tree is empty, set the root to a new node with the given value
62             self.root = Node(value)
63         else:
64             # Otherwise, call the recursive helper function to insert the value
65             self._insert_recursive(self.root, value)
66
67     def _insert_recursive(self, current_node, value):
68         """
69         Helper method to recursively insert a value into the BST.
70
71         Args:
72             current_node (Node): The current node being compared.
73             value (int): The value to be inserted.
74         """
75         if value < current_node.data:
76             # If the value is smaller than the current node's data, go to the left subtree
77             if current_node.left is None:
78                 current_node.left = Node(value)
79             else:
80                 self._insert_recursive(current_node.left, value)
81         else:
82             # If the value is greater than or equal to the current node's data, go to the right subtree
83             if current_node.right is None:
84                 current_node.right = Node(value)
85             else:
86                 self._insert_recursive(current_node.right, value)
87
88     def inorder_traversal(self):
89         """
90         Prints the elements of the BST in sorted order using an inorder traversal
91         """
92
93
94
95
96
97
98
```

Ln 18, Col 44 Spaces: 4 UTF-8 CRLF {} Python 3.14.0 Go Live Prettier

```
Binary Search Tree (BST).py X
Binary Search Tree (BST).py > ...
41 class BinarySearchTree:
96     def inorder_traversal(self):
97         """
98         Prints the elements of the BST in sorted order using an inorder traversal.
99
100        Time Complexity:
101            Best Case: O(n) - when all nodes are visited
102            Average Case: O(n) - when all nodes are visited
103            Worst Case: O(n) - when all nodes are visited
104        """
105        self._inorder_recursive(self.root)
106
107    def _inorder_recursive(self, current_node):
108        """
109        Helper method to recursively perform an inorder traversal of the BST
110        and print the values.
111        Args:
112            current_node (Node): The current node being visited.
113        """
114        if current_node is not None:
115            # Traverse the left subtree first
116            self._inorder_recursive(current_node.left)
117            # Print the current node's data
118            print(current_node.data, end=' ')
119            # Traverse the right subtree next
120            self._inorder_recursive(current_node.right)
121
122    if __name__ == "__main__":
123        #give me dynamic input for the values to be inserted into the BST
124        bst = BinarySearchTree()
125        values = input("Enter values to insert into the BST (separated by spaces): ")
126        for value in values.split():
127            bst.insert(int(value))
128        print("Inorder Traversal of the BST:")
129        bst.inorder_traversal()
```

```
Binary Search Tree (BST).py X
Binary Search Tree (BST).py > ...
41 class BinarySearchTree:
107     def _inorder_recursive(self, current_node):
109         Helper method to recursively perform an inorder traversal of the BST
110         and print the values.
111         Args:
112             current_node (Node): The current node being visited.
113         """
114         if current_node is not None:
115             # Traverse the left subtree first
116             self._inorder_recursive(current_node.left)
117             # Print the current node's data
118             print(current_node.data, end=' ')
119             # Traverse the right subtree next
120             self._inorder_recursive(current_node.right)
121
122    if __name__ == "__main__":
123        #give me dynamic input for the values to be inserted into the BST
124        bst = BinarySearchTree()
125        values = input("Enter values to insert into the BST (separated by spaces): ")
126        for value in values.split():
127            bst.insert(int(value))
128        print("Inorder Traversal of the BST:")
129        bst.inorder_traversal()
```


Output:



```
PS C:\Users\sonu\Desktop\AI_Coding> & C:/Python314/python.exe "c:/Users/sonu/Desktop/AI_Coding/Binary Search Tree (BST).py"
Enter values to insert into the BST (separated by spaces): 10 8 6 5 3 4
Inorder Traversal of the BST:
3 4 5 6 8 10
PS C:\Users\sonu\Desktop\AI_Coding> 
```

Task Description #5 – Hash Table

Task: Use AI to implement a hash table with basic insert, search, and delete methods.

Sample Input Code:

```
class HashTable: pass
```

Expected Output: • Collision handling using chaining, with well-commented methods.

Code:

Hash Table.py X

▶ ▢ ...

Hash Table.py > ...

```
1  # Act as a Python data structures expert.
2  # Generate a complete implementation of a HashTable class in Python.
3  # The hash table must:
4  # Use a fixed-size list as the underlying storage
5  # Implement a custom hash function
6  # Handle collisions using separate chaining (each index should store a list of key-value pairs)
7  # Required methods:
8  # insert(key, value) → Insert or update a key-value pair
9  # search(key) → Return the value associated with the key
10 # delete(key) → Remove the key-value pair
11 # Additional Requirements:
12 # Include proper class-level and method-level docstrings
13 # Add detailed comments explaining the hashing and collision handling logic
14 # Handle edge cases (e.g., searching or deleting a non-existent key)
15 # Follow clean coding practices
16 # Mention time complexity for each operation (Average and Worst case)
17 # Use clear variable naming
18 # Include example usage inside if __name__ == "__main__":
19 # Ensure the implementation runs correctly in VS Code
20 # Keep the code beginner-friendly but technically accurate.
21 class HashTable:
22     """
23     A HashTable implementation using separate chaining for collision resolution.
24
25     Attributes:
26         size (int): The size of the underlying storage list.
27         table (list): The list that stores key-value pairs as lists (for separate chaining).
28     """
29     def __init__(self, size=10):
30         """
31         Initializes the HashTable with a specified size.
32
33         Args:
34             size (int): The size of the hash table. Default is 10.
```

Ln 20, Col 60 Spaces: 4 UTF-8 CRLF Python 3.14.0 Go Live Prettier

```
Hash Table.py X
Hash Table.py > ...
21 class HashTable:
29     def __init__(self, size=10):
35         """
36         self.size = size
37         self.table = [[] for _ in range(size)] # Create a list of empty lists for separate chaining
38
39     def _hash(self, key):
40         """
41         Custom hash function to compute the index for a given key.
42
43         Args:
44             key: The key to be hashed.
45
46         Returns:
47             int: The computed index for the key.
48         """
49         return hash(key) % self.size # Use built-in hash and mod by size to get index
50
51     def insert(self, key, value):
52         """
53         Inserts or updates a key-value pair in the hash table.
54
55         Args:
56             key: The key to be inserted or updated.
57             value: The value associated with the key.
58
59         Time Complexity:
60             Average Case: O(1)
61             Worst Case: O(n) (when all keys hash to the same index)
62         """
63         index = self._hash(key) # Get the index for the key
64         bucket = self.table[index] # Get the bucket (list) at that index
65
66         # Check if the key already exists in the bucket
```

Ln 20, Col 60 Spaces: 4 UTF-8 CRLF {} Python 3.14.0 Go Live Prettier

```
Hash Table.py X
Hash Table.py > ...
21 class HashTable:
51     def insert(self, key, value):
65
66         # Check if the key already exists in the bucket
67         for i, (k, v) in enumerate(bucket):
68             if k == key: # If key exists, update the value
69                 bucket[i] = (key, value)
70             return
71
72         # If key does not exist, add a new key-value pair to the bucket
73         bucket.append((key, value))
74
75     def search(self, key):
76         """
77         Searches for a value associated with a given key.
78
79         Args:
80             key: The key to be searched.
81
82         Returns:
83             The value associated with the key if found, otherwise None.
84
85         Time Complexity:
86             Average Case: O(1)
87             Worst Case: O(n) (when all keys hash to the same index)
88         """
89         index = self._hash(key) # Get the index for the key
90         bucket = self.table[index] # Get the bucket (list) at that index
91
92         # Search for the key in the bucket
93         for k, v in bucket:
94             if k == key: # If key is found, return its value
95                 return v
96         return None # Return None if key is not found
```

Ln 20, Col 60 Spaces: 4 UTF-8 CRLF {} Python 3.14.0 Go Live Prettier

```
Hash Table.py X
Hash Table.py > ...
21 class HashTable:
75     def search(self, key):
95         return v
96     return None # Return None if key is not found
97
98     def delete(self, key):
99         """
100         Deletes a key-value pair from the hash table.
101
102         Args:
103             key: The key to be deleted.
104
105         Time Complexity:
106             Average Case: O(1)
107             Worst Case: O(n) (when all keys hash to the same index)
108         """
109         index = self._hash(key) # Get the index for the key
110         bucket = self.table[index] # Get the bucket (list) at that index
111
112         # Search for the key in the bucket and remove it if found
113         for i, (k, v) in enumerate(bucket):
114             if k == key: # If key is found, remove it from the bucket
115                 del bucket[i]
116             return
117         # If key is not found, do nothing (or you could raise an exception)
118
119 if __name__ == "__main__":
120     #give me dynamic input for the hash table
121     hash_table = HashTable()
122     while True:
123         action = input("Enter action (insert/search/delete/exit): ").strip().lower()
124         if action == "exit":
125             break
126         elif action in ["insert", "search", "delete"]:
```

Ln 20, Col 60 Spaces: 4 UTF-8 CRLF {} Python 3.14.0 Go Live Prettier

Task Description #6 – Graph Representation

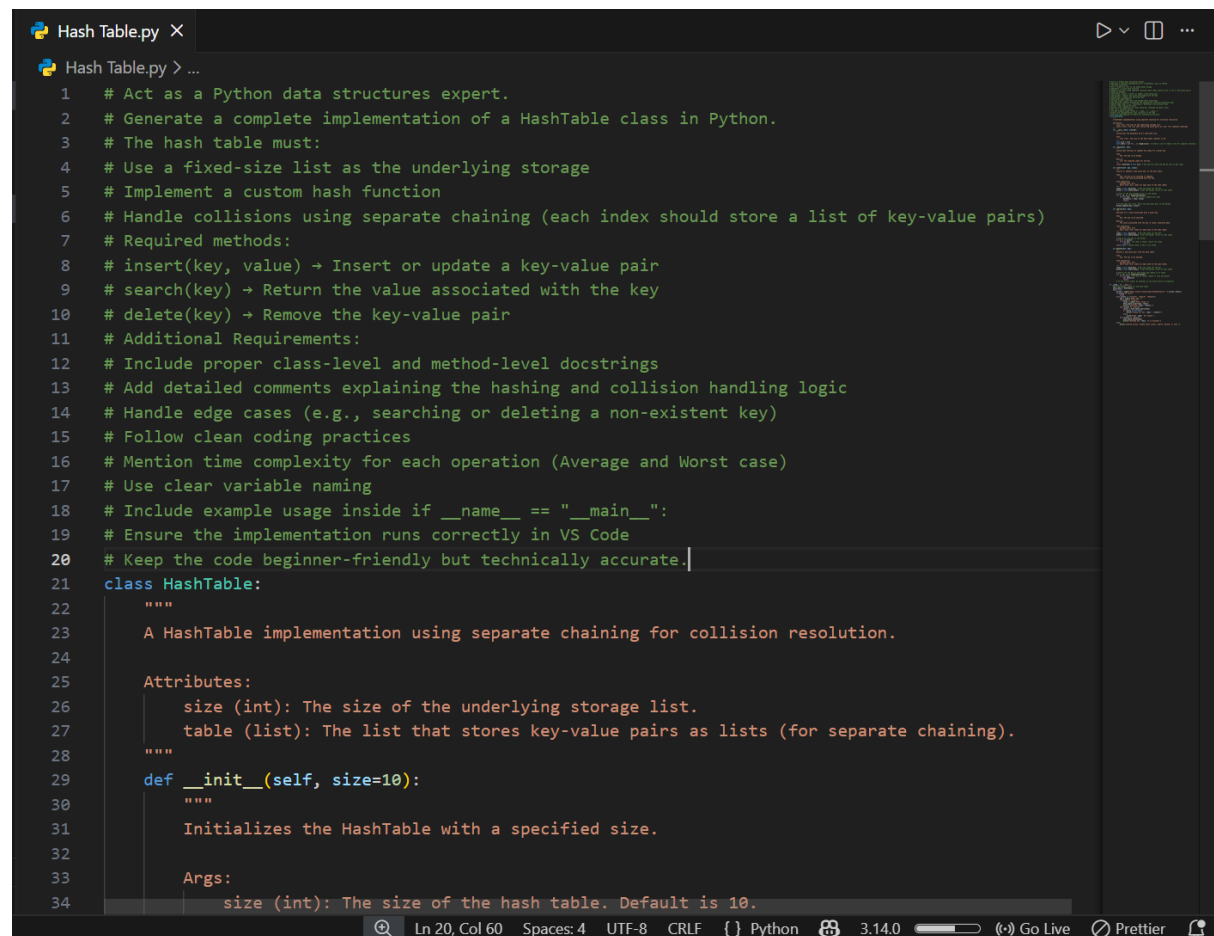
Task: Use AI to implement a graph using an adjacency list.

Sample Input Code:

```
class Graph: pass
```

Expected Output: • Graph with methods to add vertices, add edges, and display connections.

Code:



```
1  # Act as a Python data structures expert.
2  # Generate a complete implementation of a HashTable class in Python.
3  # The hash table must:
4  # Use a fixed-size list as the underlying storage
5  # Implement a custom hash function
6  # Handle collisions using separate chaining (each index should store a list of key-value pairs)
7  # Required methods:
8  # insert(key, value) → Insert or update a key-value pair
9  # search(key) → Return the value associated with the key
10 # delete(key) → Remove the key-value pair
11 # Additional Requirements:
12 # Include proper class-level and method-level docstrings
13 # Add detailed comments explaining the hashing and collision handling logic
14 # Handle edge cases (e.g., searching or deleting a non-existent key)
15 # Follow clean coding practices
16 # Mention time complexity for each operation (Average and Worst case)
17 # Use clear variable naming
18 # Include example usage inside if __name__ == "__main__":
19 # Ensure the implementation runs correctly in VS Code
20 # Keep the code beginner-friendly but technically accurate.
21 class HashTable:
22     """
23     A HashTable implementation using separate chaining for collision resolution.
24
25     Attributes:
26         size (int): The size of the underlying storage list.
27         table (list): The list that stores key-value pairs as lists (for separate chaining).
28     """
29     def __init__(self, size=10):
30         """
31         Initializes the HashTable with a specified size.
32
33         Args:
34             size (int): The size of the hash table. Default is 10.
```

```
Hash Table.py X
Hash Table.py > ...
21 class HashTable:
29     def __init__(self, size=10):
34         size (int): The size of the hash table. Default is 10.
35         """
36         self.size = size
37         self.table = [[] for _ in range(size)] # Create a list of empty lists for separate chaining
38
39     def _hash(self, key):
40         """
41         Custom hash function to compute the index for a given key.
42
43         Args:
44             key: The key to be hashed.
45
46         Returns:
47             int: The computed index for the key.
48         """
49         return hash(key) % self.size # Use built-in hash and mod by size to get index
50
51     def insert(self, key, value):
52         """
53         Inserts or updates a key-value pair in the hash table.
54
55         Args:
56             key: The key to be inserted or updated.
57             value: The value associated with the key.
58
59         Time Complexity:
60             Average Case: O(1)
61             Worst Case: O(n) (when all keys hash to the same index)
62         """
63         index = self._hash(key) # Get the index for the key
64         bucket = self.table[index] # Get the bucket (list) at that index
65
```

Ln 20, Col 60 Spaces: 4 UTF-8 CRLF {} Python 3.14.0 Go Live Prettier


```
Hash Table.py X
Hash Table.py > ...
21 class HashTable:
51     def insert(self, key, value):
65
66         # Check if the key already exists in the bucket
67         for i, (k, v) in enumerate(bucket):
68             if k == key: # If key exists, update the value
69                 bucket[i] = (key, value)
70             return
71
72         # If key does not exist, add a new key-value pair to the bucket
73         bucket.append((key, value))
74
75     def search(self, key):
76         """
77         Searches for a value associated with a given key.
78
79         Args:
80             key: The key to be searched.
81
82         Returns:
83             The value associated with the key if found, otherwise None.
84
85         Time Complexity:
86             Average Case: O(1)
87             Worst Case: O(n) (when all keys hash to the same index)
88         """
89         index = self._hash(key) # Get the index for the key
90         bucket = self.table[index] # Get the bucket (list) at that index
91
92         # Search for the key in the bucket
93         for k, v in bucket:
94             if k == key: # If key is found, return its value
95                 return v
96         return None # Return None if key is not found
```

Ln 20, Col 60 Spaces: 4 UTF-8 CRLF {} Python 3.14.0 Go Live Prettier

```
Hash Table.py X
Hash Table.py > ...
21 class HashTable:
75     def search(self, key):
95         return v
96     return None # Return None if key is not found
97
98     def delete(self, key):
99         """
100         Deletes a key-value pair from the hash table.
101
102         Args:
103             key: The key to be deleted.
104
105         Time Complexity:
106             Average Case: O(1)
107             Worst Case: O(n) (when all keys hash to the same index)
108         """
109         index = self._hash(key) # Get the index for the key
110         bucket = self.table[index] # Get the bucket (list) at that index
111
112         # Search for the key in the bucket and remove it if found
113         for i, (k, v) in enumerate(bucket):
114             if k == key: # If key is found, remove it from the bucket
115                 del bucket[i]
116                 return
117         # If key is not found, do nothing (or you could raise an exception)
118
119 if __name__ == "__main__":
120     #give me dynamic input for the hash table
121     hash_table = HashTable()
122     while True:
123         action = input("Enter action (insert/search/delete/exit): ").strip().lower()
124         if action == "exit":
125             break
126         elif action in ["insert", "search", "delete"]:
```

Ln 20, Col 60 Spaces: 4 UTF-8 CRLF Python 3.14.0 Go Live Prettier

Task Description #7 – Priority Queue

Task: Use AI to implement a priority queue using Python's heap module.

Sample Input Code:

```
class Priority Queue: pass
```

Expected Output: • Implementation with enqueue (priority), dequeue (highest priority) and display methods.

Code:

```
Priority Queue.py X
Priority Queue.py > ...
1  # Act as a Python data structures expert.
2  # Generate a complete implementation of a PriorityQueue class in Python using the built-in heapq module.
3  # The priority queue must:
4  # Use a min-heap or max-heap appropriately
5  # Store elements along with their priority
6  # Ensure that dequeue always removes the element with the highest priority
7  # Required methods:
8  # enqueue(item, priority) → Insert an element with a given priority
9  # dequeue() → Remove and return the element with the highest priority
10 # display() → Display all elements currently in the priority queue
11 # Additional Requirements:
12 # Include proper class-level and method-level docstrings
13 # Add comments explaining how the heap works internally
14 # Handle edge cases (e.g., dequeue from empty queue)
15 # Mention time complexity for enqueue and dequeue operations
16 # Follow clean coding practices
17 # Include example usage under if __name__ == "__main__":
18 # Ensure the code runs correctly in VS Code
19 # Keep the implementation beginner-friendly but technically accurate.
20 import heapq
21
22 class PriorityQueue:
23     """
24     A Priority Queue implementation using a min-heap.
25
26     This class allows you to enqueue items with a specified priority and dequeue the item with the highest priority.
27     The internal structure is a list that maintains the heap property, where the smallest element (highest priority) is at the root.
28
29     Time Complexity:
30     - Enqueue: O(log n) due to heap insertion
31     - Dequeue: O(log n) due to heap removal
32     - Display: O(n) to iterate through all elements
33     """
34     def __init__(self):
35         self.heap = []
36
37     def enqueue(self, item, priority):
38         """
39         Insert an element with a given priority.
40         """
41         self.heap.append((priority, item))
42         heapq._siftup(self.heap, len(self.heap) - 1)
43
44     def dequeue(self):
45         """
46         Remove and return the element with the highest priority.
47         """
48         if not self.heap:
49             raise ValueError("Dequeue from an empty queue")
50         priority, item = heapq._siftdown(self.heap, 0, len(self.heap) - 1)
51         self.heap[-1] = (priority, item)
52         heapq._siftup(self.heap, len(self.heap) - 1)
53         return item
54
55     def display(self):
56         """
57         Display all elements currently in the priority queue.
58         """
59         print("Elements in the priority queue (highest priority first):")
60         for priority, item in self.heap:
61             print(f"Priority: {priority}, Item: {item}")
62
63 if __name__ == "__main__":
64     pq = PriorityQueue()
65     pq.enqueue("Task 1", 1)
66     pq.enqueue("Task 2", 2)
67     pq.enqueue("Task 3", 3)
68     pq.enqueue("Task 4", 4)
69     pq.enqueue("Task 5", 5)
70     pq.display()
71     pq.dequeue()
72     pq.dequeue()
73     pq.dequeue()
74     pq.dequeue()
75     pq.dequeue()
76     pq.display()
77
78 # Example usage of the PriorityQueue class
79 # Create a priority queue
80 pq = PriorityQueue()
81
82 # Enqueue tasks with priorities
83 pq.enqueue("Task 1", 1)
84 pq.enqueue("Task 2", 2)
85 pq.enqueue("Task 3", 3)
86 pq.enqueue("Task 4", 4)
87 pq.enqueue("Task 5", 5)
88
89 # Display the queue
90 pq.display()
91
92 # Dequeue tasks (highest priority first)
93 pq.dequeue()
94 pq.dequeue()
95 pq.dequeue()
96 pq.dequeue()
97 pq.dequeue()
98
99 # Display the queue after dequeuing
100 pq.display()
```

```
Priority Queue.py X
Priority Queue.py > ...
22 class PriorityQueue:
36     def __init__(self):
37         """Initialize an empty priority queue."""
38         self._queue = [] # This list will hold the heap elements
39
40     def enqueue(self, item, priority):
41         """
42         Insert an element with a given priority into the priority queue.
43
44         Parameters:
45         item (any): The item to be added to the queue.
46         priority (int): The priority of the item. Lower values indicate higher priority.
47
48         Time Complexity: O(log n)
49         """
50         # We use a tuple (priority, item) to maintain the order in the heap
51         heapq.heappush(self._queue, (priority, item))
52
53     def dequeue(self):
54         """
55         Remove and return the element with the highest priority from the queue.
56
57         Returns:
58         any: The item with the highest priority.
59
60         Raises:
61         IndexError: If the queue is empty.
62
63         Time Complexity: O(log n)
64         """
65         if not self._queue:
66             raise IndexError("Dequeue from an empty priority queue")
67
68         # heappop removes and returns the smallest element (highest priority)
```

Ln 109, Col 13 Spaces: 4 UTF-8 CRLF Python 3.14.0 Go Live Prettier

```
Priority Queue.py X
Priority Queue.py > ...
22 class PriorityQueue:
53     def dequeue(self):
65         if not self._queue:
66             raise IndexError("Dequeue from an empty priority queue")
67
68         # heappop removes and returns the smallest element (highest priority)
69         return heapq.heappop(self._queue)[1] # Return only the item, not the priority
70
71     def display(self):
72         """
73         Display all elements currently in the priority queue along with their priorities.
74
75         Time Complexity: O(n)
76         """
77         print("Priority Queue Contents:")
78         for priority, item in self._queue:
79             print(f"Item: {item}, Priority: {priority}")
80 if __name__ == "__main__":
81     #give me dynamic input for the priority queue
82     pq = PriorityQueue()
83     while True:
84         action = input("Enter 'enqueue' to add an item, 'dequeue' to remove the highest priority item
85
86         if action == 'enqueue':
87             item = input("Enter the item to enqueue: ")
88             priority = int(input("Enter the priority (lower number means higher priority): "))
89             pq.enqueue(item, priority)
90             print(f"Enqueued '{item}' with priority {priority}.")
91
92         elif action == 'dequeue':
93             try:
94                 item = pq.dequeue()
95                 print(f"Dequeued item: {item}")
96             except IndexError as e:
```

```
Priority Queue.py X
Priority Queue.py > ...
22 class PriorityQueue:
71     def display(self):
79         print(f"Item: {item}, Priority: {priority}")
80 if __name__ == "__main__":
81     #give me dynamic input for the priority queue
82     pq = PriorityQueue()
83     while True:
84         action = input("Enter 'enqueue' to add an item, 'dequeue' to remove the highest priority item, 'display' to show the queue, or 'exit' to quit: ")
85
86         if action == 'enqueue':
87             item = input("Enter the item to enqueue: ")
88             priority = int(input("Enter the priority (lower number means higher priority): "))
89             pq.enqueue(item, priority)
90             print(f"Enqueued '{item}' with priority {priority}.")
91
92         elif action == 'dequeue':
93             try:
94                 item = pq.dequeue()
95                 print(f"Dequeued item: {item}")
96             except IndexError as e:
97                 print(e)
98
99         elif action == 'display':
100             pq.display()
101
102         elif action == 'exit':
103             print("Exiting the program.")
104             break
105
106         else:
107             print("Invalid action. Please enter 'enqueue', 'dequeue', 'display', or 'exit'.")
108
109
```

Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python + v [ ] [ ] ... [ ] [ ] X

PS C:\Users\sonu\Desktop\AI_Coding> & C:/Python314/python.exe "c:/Users/sonu/Desktop/AI_Coding/Priority Queue.py"
Enter 'enqueue' to add an item, 'dequeue' to remove the highest priority item, 'display' to show the queue, or 'exit' to q
uit: enqueue
Enter the item to enqueue: 5 6 7 8
Enter the priority (lower number means higher priority): 2
Enqueued '5 6 7 8' with priority 2.
Enter 'enqueue' to add an item, 'dequeue' to remove the highest priority item, 'display' to show the queue, or 'exit' to q
uit: display
Priority Queue Contents:
Item: 5 6 7 8, Priority: 2
Enter 'enqueue' to add an item, 'dequeue' to remove the highest priority item, 'display' to show the queue, or 'exit' to q
uit: 
```

Task Description #8 – Deque

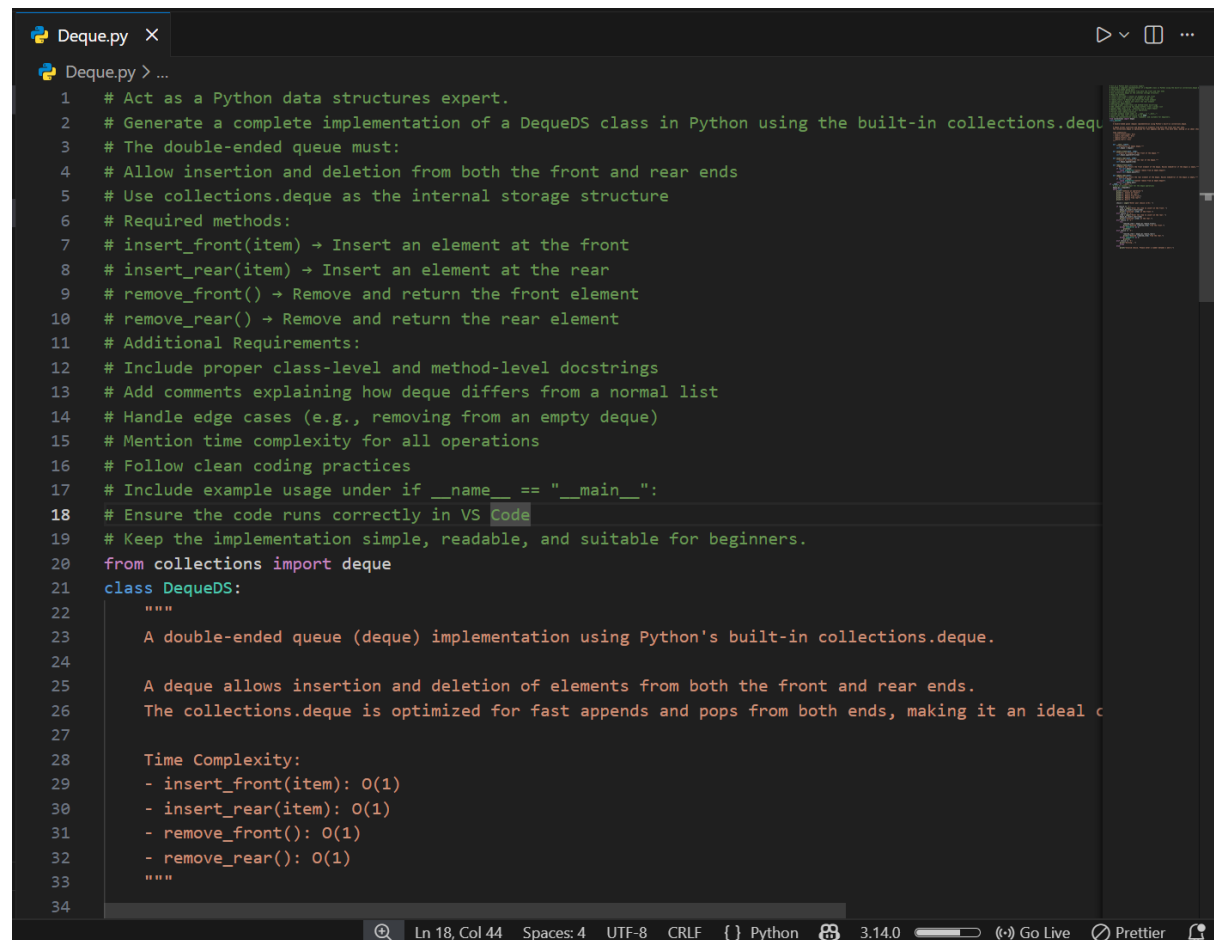
Task: Use AI to implement a double-ended queue using collections. Deque.

Sample Input Code:

```
class Deque DS : pass
```

Expected Output: • Insert and remove from both ends with docstrings.

Code:



```
1 # Act as a Python data structures expert.
2 # Generate a complete implementation of a DequeDS class in Python using the built-in collections.deque
3 # The double-ended queue must:
4 # Allow insertion and deletion from both the front and rear ends
5 # Use collections.deque as the internal storage structure
6 # Required methods:
7 # insert_front(item) → Insert an element at the front
8 # insert_rear(item) → Insert an element at the rear
9 # remove_front() → Remove and return the front element
10 # remove_rear() → Remove and return the rear element
11 # Additional Requirements:
12 # Include proper class-level and method-level docstrings
13 # Add comments explaining how deque differs from a normal list
14 # Handle edge cases (e.g., removing from an empty deque)
15 # Mention time complexity for all operations
16 # Follow clean coding practices
17 # Include example usage under if __name__ == "__main__":
18 # Ensure the code runs correctly in VS Code
19 # Keep the implementation simple, readable, and suitable for beginners.
20 from collections import deque
21 class DequeDS:
22     """
23     A double-ended queue (deque) implementation using Python's built-in collections.deque.
24
25     A deque allows insertion and deletion of elements from both the front and rear ends.
26     The collections.deque is optimized for fast appends and pops from both ends, making it an ideal c
27
28     Time Complexity:
29     - insert_front(item): O(1)
30     - insert_rear(item): O(1)
31     - remove_front(): O(1)
32     - remove_rear(): O(1)
33     """
34
```



```
Deque.py X
Deque.py > ...
21 class DequeDS:
34
35     def __init__(self):
36         """Initialize an empty deque."""
37         self.deque = deque()
38
39     def insert_front(self, item):
40         """Insert an element at the front of the deque."""
41         self.deque.appendleft(item)
42
43     def insert_rear(self, item):
44         """Insert an element at the rear of the deque."""
45         self.deque.append(item)
46
47     def remove_front(self):
48         """Remove and return the front element of the deque. Raises IndexError if the deque is empty.
49         if not self.deque:
50             raise IndexError("Cannot remove from an empty deque")
51         return self.deque.popleft()
52
53     def remove_rear(self):
54         """Remove and return the rear element of the deque. Raises IndexError if the deque is empty."
55         if not self.deque:
56             raise IndexError("Cannot remove from an empty deque")
57         return self.deque.pop()
58 if __name__ == "__main__":
59     # give me dynamic input for the deque operations
60     deque_ds = DequeDS()
61     while True:
62         print("\nChoose an operation:")
63         print("1. Insert at front")
64         print("2. Insert at rear")
65         print("3. Remove from front")
66         print("4. Remove from rear")
```

Ln 18, Col 44 Spaces: 4 UTF-8 CRLF {} Python 3.14.0 Go Live Prettier

```
Deque.py X
Deque.py > ...
60 deque_ds = DequeDS()
61 while True:
62     print("\nChoose an operation:")
63     print("1. Insert at front")
64     print("2. Insert at rear")
65     print("3. Remove from front")
66     print("4. Remove from rear")
67     print("5. Exit")
68
69     choice = input("Enter your choice (1-5): ")
70
71     if choice == '1':
72         item = input("Enter the item to insert at the front: ")
73         deque_ds.insert_front(item)
74         print(f"Inserted {item} at the front.")
75     elif choice == '2':
76         item = input("Enter the item to insert at the rear: ")
77         deque_ds.insert_rear(item)
78         print(f"Inserted {item} at the rear.")
79     elif choice == '3':
80         try:
81             removed_item = deque_ds.remove_front()
82             print(f"Removed {removed_item} from the front.")
83         except IndexError as e:
84             print(e)
85     elif choice == '4':
86         try:
87             removed_item = deque_ds.remove_rear()
88             print(f"Removed {removed_item} from the rear.")
89         except IndexError as e:
90             print(e)
91     elif choice == '5':
92         print("Exiting...")
93     break
```

Ln 18, Col 44 Spaces: 4 UTF-8 CRLF {} Python 3.14.0 Go Live Prettier

Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python + v [icon] [icon] ... [icon] [icon] X

PS C:\Users\sonu\Desktop\AI_Coding> & C:/Python314/python.exe c:/Users/sonu/Desktop/AI_Coding/Deque.py

Choose an operation:
1. Insert at front
2. Insert at rear
3. Remove from front
4. Remove from rear
5. Exit
Enter your choice (1-5): 4
Cannot remove from an empty deque

Choose an operation:
1. Insert at front
2. Insert at rear
3. Remove from front
4. Remove from rear
5. Exit
Enter your choice (1-5): 1
Enter the item to insert at the front: 3
Inserted 3 at the front.

Choose an operation:
1. Insert at front
2. Insert at rear
3. Remove from front
4. Remove from rear
5. Exit
Enter your choice (1-5): 1
Enter the item to insert at the front: 5
Inserted 5 at the front.

Choose an operation:
1. Insert at front
2. Insert at rear
3. Remove from front
4. Remove from rear
5. Exit
Enter your choice (1-5): 2
```

Ln 18, Col 44 Spaces: 4 UTF-8 CRLF { } Python 3.14.0 Go Live Prettier

- o Stack
- o Queue
- o Priority Queue
- o Linked List
- o Binary Search Tree (BST)
- o Graph
- o Hash Table
- o Deque

- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

Code:

```
Real-Time Application Challenge.py X
Real-Time Application Challenge.py > ...

9  # Graph
10 # Hash Table
11 # Deque
12 # System Features:
13 # Student Attendance Tracking - Daily log of students entering/exiting the campus.
14 # Event Registration System - Manage participants in events with quick search and removal.
15 # Library Book Borrowing - Keep track of available books and their due dates.
16 # Bus Scheduling System - Maintain bus routes and stop connections.
17 # Cafeteria Order Queue - Serve students in the order they arrive.
18 # Your Tasks:
19 # For each feature:
20 # Select the most appropriate data structure.
21 # Provide a clear justification in 2-3 sentences explaining why it fits.
22 # Mention relevant time complexity considerations.
23 # Create a well-formatted table mapping:
24 # Feature → Selected Data Structure → Justification
25 # Choose one feature and implement it as a complete working Python program with:
26 # Proper class structure
27 # Docstrings
28 # Comments explaining logic
29 # Edge case handling
30 # Time complexity notes
31 # Example usage inside if __name__ == "__main__":
32 # Ensure the implementation is beginner-friendly, efficient, and compatible with VS Code.
33 # Feature → Selected Data Structure → Justification
34 feature_mapping = [
35     ("Student Attendance Tracking", "Queue", "A queue is ideal for tracking students entering and exiting the campus."),
36     ("Event Registration System", "Hash Table", "A hash table allows for quick search, insertion, and removal of participants."),
37     ("Library Book Borrowing", "Binary Search Tree (BST)", "A BST can efficiently manage available books and their due dates."),
38     ("Bus Scheduling System", "Graph", "A graph is ideal for representing bus routes and stop connections."),
39     ("Cafeteria Order Queue", "Queue", "A queue is perfect for serving students in the order they arrive.")
40 ]
41 # Display the feature mapping in a well-formatted table
42 print(f"{'Feature':<30} {'Selected Data Structure':<25} {'Justification':<40}")
```

```
Real-Time Application Challenge.py X
Real-Time Application Challenge.py > ...
40 ]
41 # Display the feature mapping in a well-formatted table
42 print(f"{'Feature':<30} {'Selected Data Structure':<25} {'Justification'}")
43 print("-" * 80)
44 for feature, data_structure, justification in feature_mapping:
45     print(f"{'feature':<30} {'data_structure':<25} {'justification'}")
46 # Implementing the Cafeteria Order Queue feature using a Queue data structure
47 class CafeteriaOrderQueue:
48     """
49     A class to represent a cafeteria order queue.
50     This class uses a list to implement the queue functionality.
51     """
52
53     def __init__(self):
54         """Initialize an empty queue."""
55         self.queue = []
56
57     def enqueue(self, order):
58         """
59         Add an order to the end of the queue.
60
61         :param order: The order to be added (string).
62         """
63         self.queue.append(order) # O(1) time complexity
64
65     def dequeue(self):
66         """
67         Remove and return the order at the front of the queue.
68
69         :return: The order at the front of the queue (string) or None if the queue is empty.
70         """
71         if not self.is_empty():
72             return self.queue.pop(0) # O(n) time complexity due to shifting elements
73         else:
```

Ln 32, Col 90 Spaces: 4 UTF-8 CRLF {} Python 3.14.0 Go Live Prettier

```
Real-Time Application Challenge.py X
Real-Time Application Challenge.py > ...
47 class CafeteriaOrderQueue:
65     def dequeue(self):
69         :return: The order at the front of the queue (string) or None if the queue is empty.
70         """
71         if not self.is_empty():
72             return self.queue.pop(0) # O(n) time complexity due to shifting elements
73         else:
74             print("The queue is empty. No orders to serve.")
75             return None
76
77     def is_empty(self):
78         """
79         Check if the queue is empty.
80
81         :return: True if the queue is empty, False otherwise.
82         """
83         return len(self.queue) == 0
84
85     def size(self):
86         """
87         Get the number of orders in the queue.
88
89         :return: The size of the queue (integer).
90         """
91         return len(self.queue)
92
93 if __name__ == "__main__":
94     #give me dynamic input for the cafeteria order queue
95     cafeteria_queue = CafeteriaOrderQueue()
96     while True:
97         action = input("Enter 'enqueue' to add an order, 'dequeue' to serve an order, or 'exit' to qu
98         if action == 'enqueue':
99             order = input("Enter the order to be added: ")
100             cafeteria_queue.enqueue(order)
101             print(f"Order '{order}' added to the queue.")
```

```
Real-Time Application Challenge.py X
Real-Time Application Challenge.py > ...
47 class CafeteriaOrderQueue:
85     def size(self):
87         Get the number of orders in the queue.
88
89         :return: The size of the queue (integer).
90         """
91         return len(self.queue)
92
93 if __name__ == "__main__":
94     #give me dynamic input for the cafeteria order queue
95     cafeteria_queue = CafeteriaOrderQueue()
96     while True:
97         action = input("Enter 'enqueue' to add an order, 'dequeue' to serve an order, or 'exit' to qu
98         if action == 'enqueue':
99             order = input("Enter the order to be added: ")
100             cafeteria_queue.enqueue(order)
101             print(f"Order '{order}' added to the queue.")
102         elif action == 'dequeue':
103             served_order = cafeteria_queue.dequeue()
104             if served_order:
105                 print(f"Order '{served_order}' has been served.")
106         elif action == 'exit':
107             print("Exiting the cafeteria order queue system.")
108             break
109         else:
110             print("Invalid action. Please enter 'enqueue', 'dequeue', or 'exit'.")
```


Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\sonu\Desktop\AI_Coding> & C:/Python314/python.exe "c:/Users/sonu/Desktop/AI_Coding/Real-Time Application Challenge.py"
Feature                               Selected Data Structure  Justification
-----
Student Attendance Tracking           Queue                   A queue is ideal for tracking students entering and exiting the campus as it follows the First-In-First-Out (FIFO) principle, allowing for efficient logging of attendance. Time complexity for enqueue and dequeue operations is O(1).
Event Registration System              Hash Table              A hash table allows for quick search, insertion, and deletion of participants in events, making it suitable for managing registrations. Time complexity for search, insert, and delete operations is O(1) on average.
Library Book Borrowing                 Binary Search Tree (BST) A BST can efficiently manage available books and their due dates by allowing for sorted data storage and quick retrieval. Time complexity for search, insert, and delete operations is O(log n) on average.
Bus Scheduling System                  Graph                   A graph is ideal for representing bus routes and stop connections as it can model complex relationships between stops and routes. Time complexity for traversal operations is O(V + E), where V is the number of vertices (stops) and E is the number of edges (routes).
Cafeteria Order Queue                  Queue                   A queue is perfect for serving students in the order they arrive at the cafeteria, ensuring fairness. Time complexity for enqueue and dequeue operations is O(1).
Enter 'enqueue' to add an order, 'dequeue' to serve an order, or 'exit' to quit: enqueue
Enter the order to be added: 5 7 9 10
Order '5 7 9 10' added to the queue.
Enter 'enqueue' to add an order, 'dequeue' to serve an order, or 'exit' to quit: dequeue
Order '5 7 9 10' has been served.
Enter 'enqueue' to add an order, 'dequeue' to serve an order, or 'exit' to quit: exit
Exiting the cafeteria order queue system.
PS C:\Users\sonu\Desktop\AI_Coding>
```

Task Description #10: Smart E-Commerce Platform – Data Structure Challenge

An e-commerce company wants to build a Smart Online Shopping System with:

1. Shopping Cart Management – Add and remove products dynamically.
2. Order Processing System – Orders processed in the order they are placed.
3. Top-Selling Products Tracker – Products ranked by sales count.
4. Product Search Engine – Fast lookup of products using product ID.
5. Delivery Route Planning – Connect warehouses and delivery locations.

Student Task:

- For each feature, select the most appropriate data structure from the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

Code:

```
Smart E-Commerce Platform.py X
Smart E-Commerce Platform.py > ...
1  # Act as a data structures expert and system architect.
2  # Analyze the following Smart E-Commerce Platform requirements and select the most appropriate data s
3  # Available Data Structures:
4  # Stack
5  # Queue
6  # Priority Queue
7  # Linked List
8  # Binary Search Tree (BST)
9  # Graph
10 # Hash Table
11 # Deque
12 # System Features:
13 # Shopping Cart Management - Add and remove products dynamically.
14 # Order Processing System - Orders processed in the order they are placed.
15 # Top-Selling Products Tracker - Products ranked by sales count.
16 # Product Search Engine - Fast lookup of products using product ID.
17 # Delivery Route Planning - Connect warehouses and delivery locations.
18 # Required Tasks:
19 # For each feature:
20 # Select the most appropriate data structure.
21 # Provide a clear 2-3 sentence justification.
22 # Mention relevant time complexity considerations.
23 # Present the results in a well-formatted table:
24 # Feature -> Selected Data Structure -> Justification
25 # Choose one feature and implement it as a complete Python program with:
26 # Proper class-based structure
27 # Docstrings for class and methods
28 # Inline comments explaining logic
29 # Edge case handling
30 # Time complexity notes
31 # Example usage under if __name__ == "__main__":
32 # Code compatible with VS Code
33 # Ensure the solution is technically accurate, beginner-friendly, and follows clean coding practices.
34 # Feature -> Selected Data Structure -> Justification

Ln 31, Col 49  Spaces: 4  UTF-8  CRLF  {} Python  3.14.0  Go Live  Prettier
```

```
Smart E-Commerce Platform .py X
Smart E-Commerce Platform.py > ...
33 # Ensure the solution is technically accurate, beginner-friendly, and follows clean coding practices.
34 # Feature → Selected Data Structure → Justification
35 features_data_structures = [
36     ("Shopping Cart Management", "Linked List", "A linked list allows for dynamic addition and removal of items in the cart, making it suitable for managing a shopping cart efficiently."),
37     ("Order Processing System", "Queue", "A queue is ideal for processing orders in the order they are received, ensuring fairness and preventing starvation."),
38     ("Top-Selling Products Tracker", "Priority Queue", "A priority queue can efficiently keep track of top-selling products by prioritizing them based on sales volume."),
39     ("Product Search Engine", "Hash Table", "A hash table allows for fast lookup of products using product IDs or names, reducing search time significantly."),
40     ("Delivery Route Planning", "Graph", "A graph is suitable for modeling warehouses and delivery locations, enabling efficient route planning algorithms."),
41 ]
42 # Display the results in a well-formatted table
43 print(f'{"Feature":<30} {"Selected Data Structure":<25} {"Justification"}')
44 for feature, data_structure, justification in features_data_structures:
45     print(f'{"feature":<30} {"data_structure":<25} {"justification"}')
46 # Implementing the Shopping Cart Management feature using a Linked List
47 class Node:
48     """A node in the linked list representing a product in the shopping cart."""
49     def __init__(self, product_id, quantity):
50         self.product_id = product_id
51         self.quantity = quantity
52         self.next = None
53 class ShoppingCart:
54     """A shopping cart implemented as a linked list."""
55     def __init__(self):
56         self.head = None
57     def add_product(self, product_id, quantity):
58         """Add a product to the shopping cart."""
59         new_node = Node(product_id, quantity)
60         new_node.next = self.head
61         self.head = new_node
62     def remove_product(self, product_id):
63         """Remove a product from the shopping cart."""
64         current = self.head
65         previous = None
66         while current is not None:
```


Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\sonu\Desktop\AI_Coding> & C:/Python314/python.exe "c:/Users/sonu/Desktop/AI_Coding/Smart E-Commerce Platform .py"
Feature                               Selected Data Structure   Justification
Shopping Cart Management              Linked List               A linked list allows for dynamic addition and removal of products
without needing to shift elements, making it efficient for managing a shopping cart. Time complexity for adding/removing
products is O(1).
Order Processing System               Queue                    A queue is ideal for processing orders in the order they are plac
ed (FIFO). Time complexity for enqueue and dequeue operations is O(1).
Top-Selling Products Tracker          Priority Queue            A priority queue can efficiently keep track of top-selling produc
ts by maintaining a sorted order based on sales count. Time complexity for insertion and retrieval is O(log n).
Product Search Engine                Hash Table               A hash table allows for fast lookup of products using product ID
with an average time complexity of O(1) for search operations. It also handles collisions effectively.
Delivery Route Planning              Graph                    A graph is suitable for modeling warehouses and delivery location
s, allowing for efficient route planning using algorithms like Dijkstra's or A*. Time complexity depends on the algorithm
used, typically O(V + E) for Dijkstra's algorithm.
Enter 'add' to add a product, 'remove' to remove a product, 'display' to show cart contents, or 'exit' to quit: add
Enter product ID: 567903
Enter quantity: 300
Added Product ID: 567903 with Quantity: 300
Enter 'add' to add a product, 'remove' to remove a product, 'display' to show cart contents, or 'exit' to quit: display
Shopping Cart Contents:
Product ID: 567903, Quantity: 300
Enter 'add' to add a product, 'remove' to remove a product, 'display' to show cart contents, or 'exit' to quit: 
```